Estun Codroid 使用问题手册

1.手册

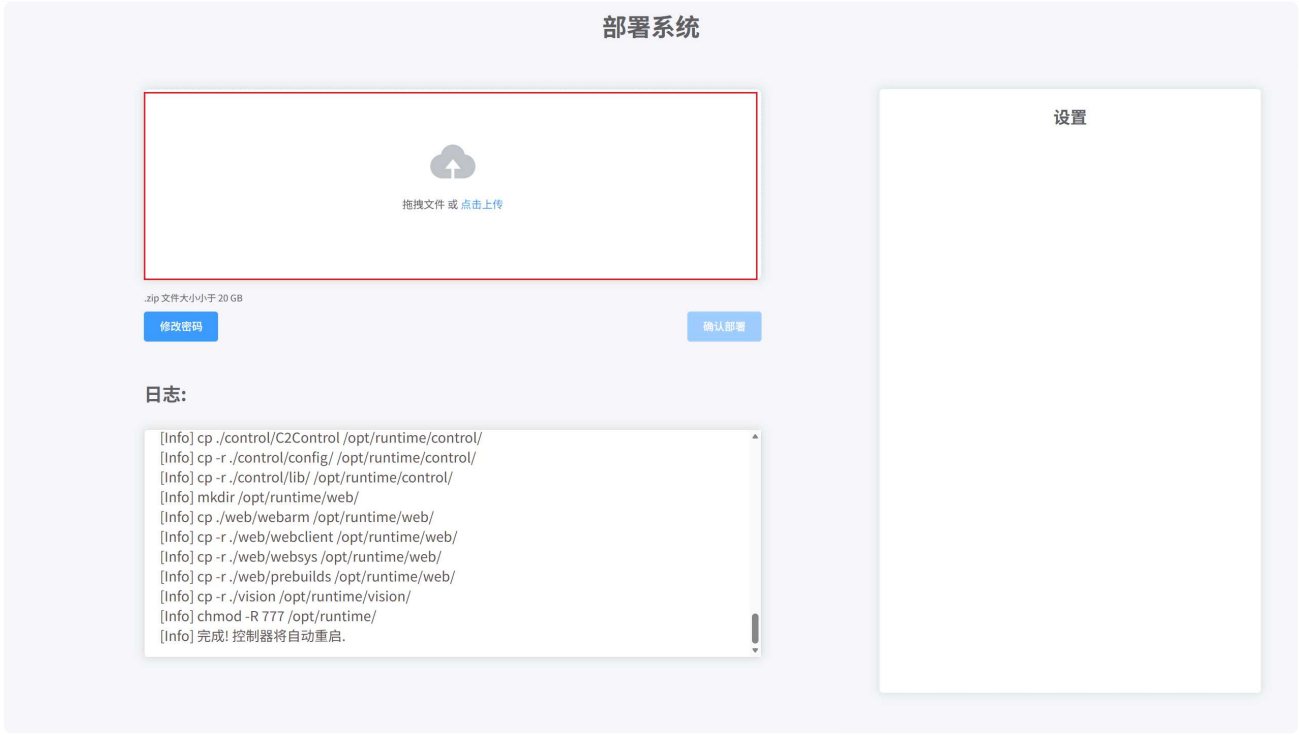
寄存器协议Modbusv2.3(20260114版).xlsx
File size:21.26 KB

2.前端升级

2.1 升级前确保能连上上位机界面(默认网址 192.168.1.136:9198)



2.2 进入上位机部署界面(默认网址 192.168.1.136:8080)



2.3 把技术人员发送的升级包下载下来并拖拽至上图红框内(请勿解压)，等待升级包上传并解析完成

2.4 请详细查看设置选项说明，并在日志记录里查看本机配置信息(软件类型，机器人型号)



2.5 在右侧设置内选择对应的选项(如无特殊说明，一般为更新，通用，相应的机器人型号)

2.6 点击确认部署按钮

2.7 输入密码(密码:estun)，并点击确认按钮

2.8 机器人控制器会自动重启，等待机器人重启完毕即可

3.固件升级

3.1 升级前确保能连上上位机界面(默认网址 192.168.1.136:9198)



3.2 输入账号密码登录上位机

账号：admin

密码：123456

3.3 确认机器人处于未使能状态，依次点击应用 -> 部署，



3.4 将技术人员给的升级包拖拽至红框内(通常情况为 .bin 结尾的二进制文件)



3.5 根据上传的升级包选择对应的升级，如上图所示是 MSC 部署包(通常部署包名会带有相应的简写，如 MSCZLG06.bin 为 MSC 升级，只能勾选 MSC 从站，MFC 同理，Joint 需要勾选 joint1~joint6)

3.6 勾选上 MSC 从站，点击确认部署按钮，输入密码(密码 :codroidsafety)

3.7 此时 web 界面会弹出正在升级的报警弹窗，等待提示升级完成，手动重启控制柜

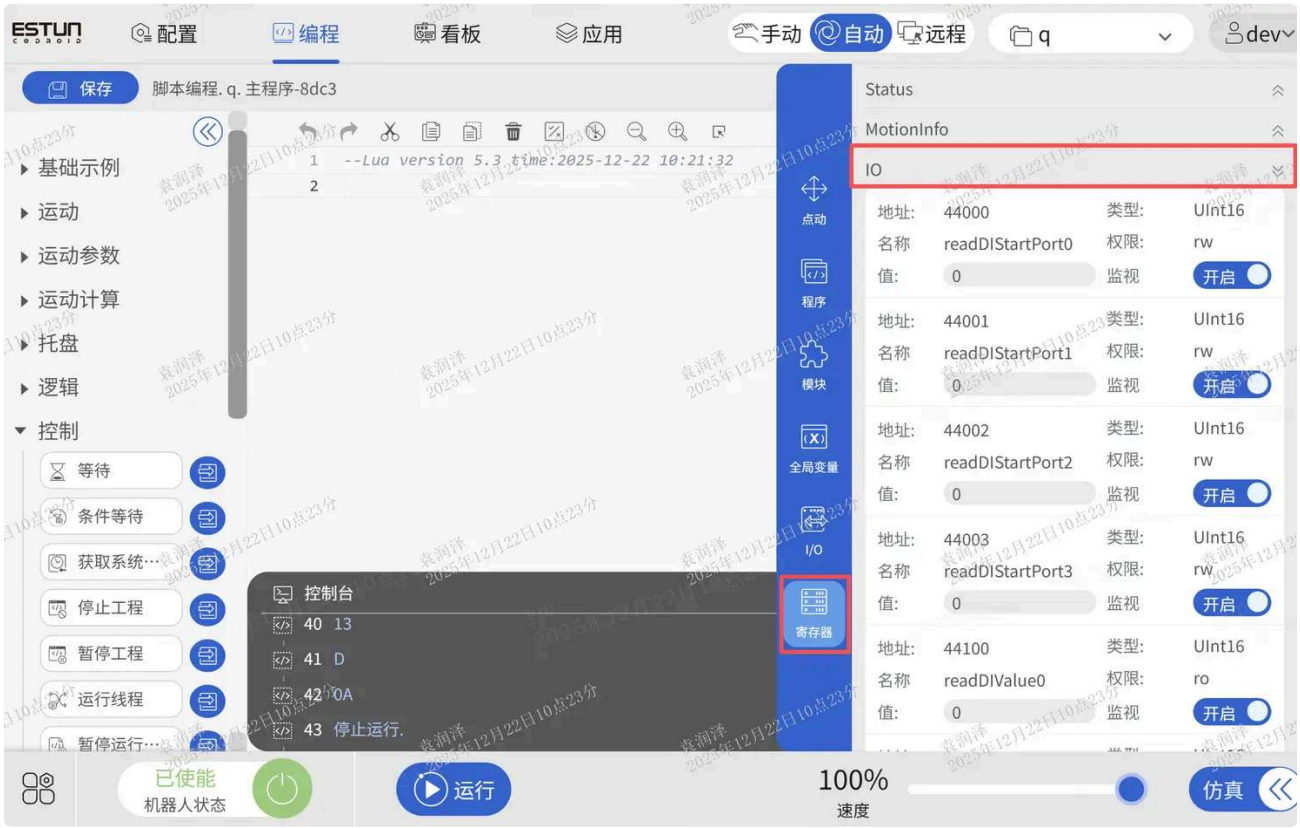
4.寄存器

以下只包含寄存器复合使用手册，单独寄存器读取设置注释请参考 ModbusTcp 从站手册

4.1 读取 DI

找到寄存器 IO 栏 readDIStartPort 和 readDIValue，为一组配对使用(地址 44000~44003，44100~44103)

以 readDIStartPort0 和 readDIValue0 为例，readDIStartPort 表示从第 DI 几开始到第 DI15，readDIValue，则为读取的 DI 数据的 10 进制值。

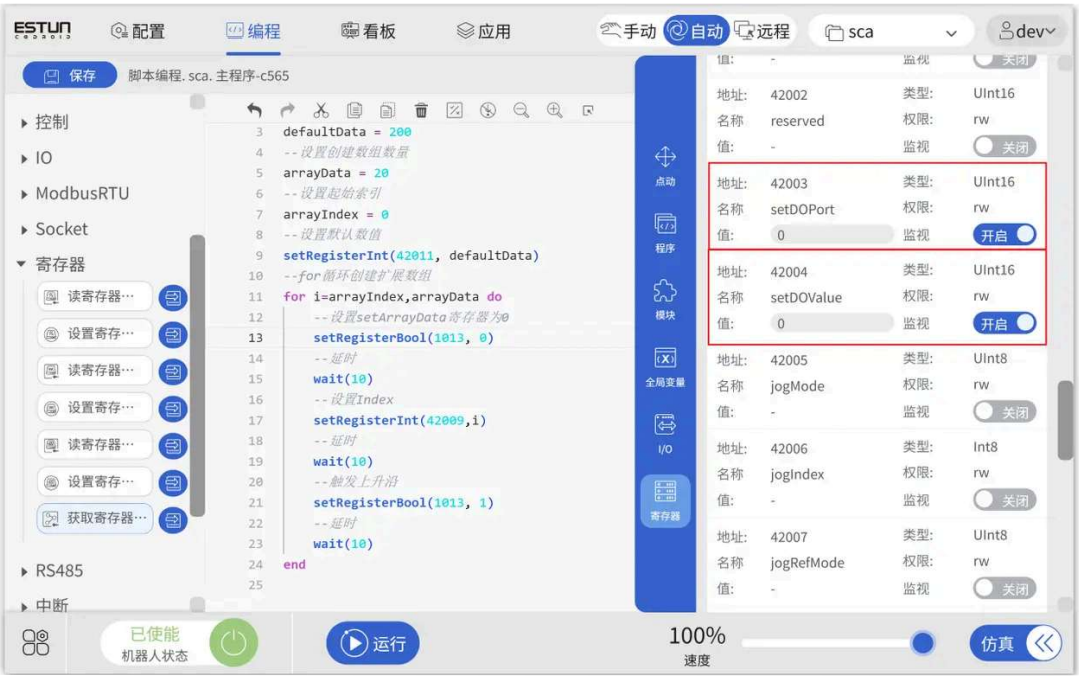


示例：测试情况输入 IO 为 DI0, DI2, DI4, DI6, DI8, DI10, DI12, DI14 为开

readDIStartPort	readDIValue	含义
0	21845	0101010101010101，从右至左依次是 DI0~DI15
3	2730	0101010101010，从右至左依次是 DI3~DI15，位数不
7	170	010101010，从右至左依次是 DI7~DI15，位数不足记
15	0	0，DI15

4.2 设置 DO

找到寄存器 Control 栏 setDOPort 和 setDOValue，为一组配对使用(地址 42003，42004)



每次使用时需要先将 setDOPort(42003)传入 **65535**，再设置 setDOValue(42004)为目标值(除了 0 之外的所有值均视为 1)，最后将 setDOPort(42003)设置为需要更改的 DO 端口号(取值 0~15)

4.3 寄存器控制启动指定程序

1. 配置工程映射，按照指示依次点击配置 ->IO-> 工程映射



2. 将需要指定的程序选择工程映射并**点击保存**



3. **注意：可视化程序不会出现在列表内，需要点击转换成脚本后，对脚本程序进行工程映射操作**



4. 打开寄存器界面，选择 control 标签栏



5. 找到 **startProject** 和 **startProjectNumber** 寄存器(默认地址是 1000 和 42000)





6. 在 **startProjectNumber** 寄存器写入需要启动的程序索引，然后在 **startProject** 寄存器写入 0，再写入 1，即可启动对应的程序(**startProject** 寄存器为**上升沿触发**)

4.4 扩展寄存器使用

1. 打开寄存器界面，选择 control 标签栏



2. 找到 **setArrayData**、**arrayIndex**、**arrayData** 三个寄存器（默认地址：1013,42009,42011）





3. **arrayIndex** 设置寄存器索引编号

arrayData 设置寄存器初始值(目前初始值只能设置**整数类型**)

setArrayData 添加扩展数组(**上升沿触发**)

4. 先进行如下流程，arrayIndex 下发 0，arrayData 下发 200，setArrayData 下发 0，然后下发 1(触发上升沿), 查看寄存器界面的 **ExtendArray** 标签栏



5. 查看多出来的扩展数组，类型可以按需更改，**创建时只能默认 Int32**

创建地址按照 50000+index*2 计算，该地址可被 ModbusTcp 主站访问，但**不能被 PN 访问**

6. 连续创建 20 个扩展数组例程

Lua

```lua
--创建时默认数据类型为Int32
--设置默认值
defaultData = 200
--设置创建数组数量
arrayData = 20
--设置起始索引
arrayIndex = 0
--设置默认数值
setRegisterInt(42011, defaultData)
--for循环创建扩展数组
for i=arrayIndex,arrayData do
    --设置setArrayData寄存器为0
    setRegisterBool(1013, 0)
    --延时
    wait(10)
    --设置Index
    setRegisterInt(42009,i)
    --延时
    wait(10)
    --触发上升沿
    setRegisterBool(1013, 1)
    --延时
    wait(10)
end

```

4.5 寄存器控制机器人点动

1. 打开寄存器面板，control 标签栏，找到 **startJog** 寄存器(1012)



jogMode, **jogIndex**, **jogRefMode** (42005,42006,42007)



Base 标签栏找到 **heartBeatFromMaster** 寄存器 (41011)



2. **heartBeatFromMaster** 为点动心跳，需要以每 500ms 以内的速率向改寄存器发送递增数字，**jogMode** 设置点动模式(1: 关节电动，2: 笛卡尔电动)

jogIndex 设置索引值(取值范围±1~6，正负代表方向，1~6 代表关节 1~6，或者{x, y, z, rx, ry, rz})

jogRefMode 笛卡尔点动时生效，设置点动坐标系类型(0：用户坐标系，1：工具坐标系)

startJog 启动点动(**上升沿触发点动，下降沿停止点动**)

3. 心跳发送代码

testModbus.py.txt
File size:10.72 KB

5. 关节序列号不匹配问题

1. dev 权限登录上位机，开机后会自动读取一次实际 6 个关节序列号和当前设定的 6 个关节序列号是否一致，不一致则会弹窗提示



2. 进入【日志】，从报错日志中可以查看所有关节的实际序列号（可以点击【下载日志】按钮，方便复制粘贴），然后将读取到的序列号填入【生产 - 基本信息】中的关节序列号 1~6 中



3. 填写完成后点击左下角【应用】按钮，密码 codroidsafety



示例程序：

1. 字节流传输(客户端)

Lua

```lua
--目前只能使用单一类型的字节流传输，不能使用自定义多种类型结构体
--发送数据写十进制数组(表结构)即可，发送类型在参数中以字符串形式传入
--接收数据存储在data变量，会根据接受数据类型自动转换为10进制数组(表结构)

--测试使用网络调试助手，收发都改成Hex格式

--创建客户端
--   参数1：客户端名称
--   参数2：服务器IP
--   参数3：服务器端口号
res = createSocketClient("client","192.168.1.166",8080)
--连接服务器
--   参数1：客户端名称
--   参数2：连接超时时间
res = connectSocketClient("client",3000)
--写入字节流
--   参数1：客户端名称
--   参数2：发送数据，必须是table类型
--   参数3：数据类型可:{Int8,UInt8,Int16,UInt16,Int32,UInt32,Float,Double}
--   参数4：是否为大端，true为大端，false为小端
res = writeByteSocketClient("client",{19.87},"Float",false)

send_data = {19.27,89.34,768,78,345.987}
res = writeByteSocketClient("client",send_data,"Float",false)

--读取字节流
--   参数1：客户端名称
--   参数2：数据类型可:{Int8,UInt8,Int16,UInt16,Int32,UInt32,Float,Double}
--   参数3：是否为大端，true为大端，false为小端
--   参数4：接收超时时间
--   返回值 data为table类型
res,data = readByteSocketClient("client","Float",false,30000)
print(data)


```

2. 字节流传输(服务端)

Lua

```lua
--目前只能使用单一类型的字节流传输，不能使用自定义多种类型结构体
--发送数据写十进制数组(表结构)即可，发送类型在参数中以字符串形式传入
--接收数据存储在data变量，会根据接受数据类型自动转换为10进制数组(表结构)

--测试使用网络调试助手，收发都改成Hex格式

--创建客户端
--   参数1：服务端名称
--   参数3：服务端端口号(20000~65535)
res = createSocketServer("server",23456)
--等待客户端连接
--   参数1：服务端名称
--   参数2：连接超时时间(写大一点)
res = waitConnectSocketServer("server",30000000000000)
--写入字节流
--   参数1：服务端名称
--   参数2：发送数据，必须是table类型
--   参数3：数据类型可:{Int8,UInt8,Int16,UInt16,Int32,UInt32,Float,Double}
--   参数4：是否为大端，true为大端，false为小端

-- res = writeByteSocketServer("server",{19.87},"Float",false)
send_data = {19.27,89.34,768,78,345.987}
res = writeByteSocketServer("server",send_data,"Float",false)

--读取字节流
--   参数1：服务端名称
--   参数2：数据类型可:{Int8,UInt8,Int16,UInt16,Int32,UInt32,Float,Double}
--   参数3：是否为大端，true为大端，false为小端
--   参数4：接收超时时间
--   返回值 data为table类型
res,data = readByteSocketServer("server","Float",false,30000000)
print(data)


```